

String-2 > double_char

[prev](#) | [next](#) | [chance](#)

Given a string, return a string where for every char in the original, there are two chars.

double_char('The') → 'TThhee'
double_char('AAbb') → 'AAAAbbbb'
double_char('Hi-There') → 'HHii--TThheerree'

Go

...Save, Compile, Run (ctrl-enter)

Show Hint

```
def double_char(str):  
    res = ""  
    for char in str:  
        res += char * 2  
    return res
```

Expected	Run		
double_char('The') → 'TThhee'	'TThhee'	OK	
double_char('AAbb') → 'AAAAbbbb'	'AAAAbbbb'	OK	
double_char('Hi-There') → 'HHii--TThheerree'	'HHii--TThheerree'	OK	
double_char('Word!') → 'WWoorrrd!!'	'WWoorrrd!!'	OK	
double_char('!!!') → '!!!!'	'!!!!'	OK	
double_char('') → ''	''	OK	
double_char('a') → 'aa'	'aa'	OK	
double_char('.') → '..'	'..'	OK	
double_char('aa') → 'aaaa'	'aaaa'	OK	
other tests		OK	

 All Correct

Good job -- problem solved. You can see our solution as an alternative.

[See Our Solution](#)

String-2 > count_hi

[prev](#) | [next](#) | [chance](#)

Return the number of times that the string "hi" appears anywhere in the given string.

count_hi('abc hi ho') → 1

count_hi('ABChi hi') → 2

count_hi('hihi') → 2

Go

...Save, Compile, Run (ctrl-enter)

Show Hint

```
def count_hi(str):  
    return str.count('hi')
```

Expected

Run

count_hi('abc hi ho') → 1	1	OK	
count_hi('ABChi hi') → 2	2	OK	
count_hi('hihi') → 2	2	OK	
count_hi('hiHIhi') → 2	2	OK	
count_hi('') → 0	0	OK	
count_hi('h') → 0	0	OK	
count_hi('hi') → 1	1	OK	
count_hi('Hi is no HI on ahI') → 0	0	OK	
count_hi('hiho not HOHIhi') → 2	2	OK	
other tests		OK	



All Correct

Good job -- problem solved. You can see our solution as an

String-2 > cat_dog

[prev](#) | [next](#) | [chance](#)

Return True if the string "cat" and "dog" appear the same number of times in the given string.

cat_dog('catdog') → True

cat_dog('catcat') → False

cat_dog('1cat1cadodog') → True

Go

...Save, Compile, Run (ctrl-enter)

```
def cat_dog(str):  
    return str.count('cat') == str.count('dog')
```

Go

Expected

Run

cat_dog('catdog') → True	True	OK	
cat_dog('catcat') → False	False	OK	
cat_dog('1cat1cadodog') → True	True	OK	
cat_dog('catxxdogxxxdog') → False	False	OK	
cat_dog('catxdogxdogxcat') → True	True	OK	
cat_dog('catxdogxdogxca') → False	False	OK	
cat_dog('dogdogcat') → False	False	OK	
cat_dog('dogogcat') → True	True	OK	
cat_dog('dog') → False	False	OK	
cat_dog('cat') → False	False	OK	
cat_dog('ca') → True	True	OK	
cat_dog('c') → True	True	OK	
cat_dog('') → True	True	OK	
other tests		OK	



All Correct

String-2 > count_code

[prev](#) | [next](#) | [chance](#)

Return the number of times that the string "code" appears anywhere in the given string, except we'll accept any letter for the 'd', so "cope" and "cooe" count.

count_code('aaacodebbb') → 1
count_code('codexxcode') → 2
count_code('cozexxcope') → 2

Go

...Save, Compile, Run (ctrl-enter)

```
def count_code(str):  
    count = 0  
    for i in range(len(str) - 3):  
        if str[i:i+2] == 'co' and str[i+3] == 'e':  
            count += 1  
    return count
```

Go

Expected	Run		
count_code('aaacodebbb') → 1	1	OK	
count_code('codexxcode') → 2	2	OK	
count_code('cozexxcope') → 2	2	OK	
count_code('cozfxxcop') → 1	1	OK	
count_code('xxcozeyycop') → 1	1	OK	
count_code('cozcop') → 0	0	OK	
count_code('abcxyz') → 0	0	OK	
count_code('code') → 1	1	OK	
count_code('ode') → 0	0	OK	
count_code('c') → 0	0	OK	
count_code('') → 0	0	OK	
count_code('AAcodeBBcoleCCcocreDD') → 3	3	OK	
count_code('AAcodeBBcoleCCccorfeDD') → 2	2	OK	
count_code('coAcodeBcoleccocreDD') → 3	3	OK	
other tests		OK	



All Correct

String-2 > end_other

[prev](#) | [next](#) | [chance](#)

Given two strings, return True if either of the strings appears at the very end of the other string, ignoring upper/lower case differences (in other words, the computation should not be "case sensitive"). Note: s.lower() returns the lowercase version of a string.

end_other('Hiabc', 'abc') → True
end_other('AbC', 'HiaBc') → True
end_other('abc', 'abXabc') → True

Go

...Save, Compile, Run (ctrl-enter)

Show Hint

```
def end_other(a, b):  
    a = a.lower()  
    b = b.lower()  
    return a.endswith(b) or b.endswith(a)
```

Expected	Run		
end_other('Hiabc', 'abc') → True	True	OK	
end_other('AbC', 'HiaBc') → True	True	OK	
end_other('abc', 'abXabc') → True	True	OK	
end_other('Hiabc', 'abcd') → False	False	OK	
end_other('Hiabc', 'bc') → True	True	OK	
end_other('Hiabcx', 'bc') → False	False	OK	
end_other('abc', 'abc') → True	True	OK	
end_other('xyz', '12xyz') → True	True	OK	
end_other('yz', '12xz') → False	False	OK	
end_other('Z', '12xz') → True	True	OK	
end_other('12', '12') → True	True	OK	
end_other('abcXYZ', 'abcDEF') → False	False	OK	
end_other('ab', 'ab12') → False	False	OK	
end_other('ab', '12ab') → True	True	OK	
other tests		OK	



All Correct

String-2 > xyz_there

[prev](#) | [next](#) | [chance](#)

Return True if the given string contains an appearance of "xyz" where the xyz is not directly preceeded by a period (.). So "xxyz" counts but "x.xyz" does not.

xyz_there('abcxyz') → True

xyz_there('abc.xyz') → False

xyz_there('xyz.abc') → True

Go

...Save, Compile, Run (ctrl-enter)

```
def xyz_there(str):
    for i in range(len(str) - 2):
        if str[i:i+3] == 'xyz':
            if i == 0 or str[i-1] != '.':
                return True
    return False
```

Go

Expected	Run		
xyz_there('abcxyz') → True	True	OK	
xyz_there('abc.xyz') → False	False	OK	
xyz_there('xyz.abc') → True	True	OK	
xyz_there('abcxy') → False	False	OK	
xyz_there('xyz') → True	True	OK	
xyz_there('xy') → False	False	OK	
xyz_there('x') → False	False	OK	
xyz_there('') → False	False	OK	
xyz_there('abc.xyzxyz') → True	True	OK	
xyz_there('abc.xxyz') → True	True	OK	
xyz_there('.xyz') → False	False	OK	
xyz_there('12.xyz') → False	False	OK	
xyz_there('12xyz') → True	True	OK	
xyz_there('1.xyz.xyz2.xyz') → False	False	OK	
other tests		OK	



All Correct